

ZDRAVO JEM

Project Documentation

Technical, deployment, operations, and maintenance guide

Version 2.0

Prepared 16 August 2026

Installable PWA for a 55-inch portrait touchscreen kiosk

Slovenian and English interface

Secret-safe documentation: no keys, passwords, or tokens included

Version 2.0 Prepared: 16 August 2026

Zdravo Jem ("I Eat Healthy") is an installable Progressive Web App designed for a 55-inch portrait touchscreen kiosk at a Slovenian farmers' market. It helps visitors discover recipes from market ingredients, view cooking guidance, send recipes by email, and learn through two food-themed games.

This document is intended for project owners, developers, content administrators, deployment administrators, and kiosk support staff.

1. Product overview

The kiosk provides the following visitor experiences:

- A Slovenian and English interface selected with the SL/EN control.
- Ingredient and category browsing.
- Recipe recommendations based on selected ingredients.
- Recipe details including description, ingredients, preparation steps, additional advice, time, servings, and difficulty.
- Email and QR-code sharing.
- The "Od kmetije do kroznika" Farm to Plate puzzle game.
- The Market Detective question game.
- Offline operation after the application and catalogue have been cached.

The application was migrated from Electron and local SQLite to a browser-based PWA. The current frontend uses plain JavaScript ES modules and reads catalogue data from Supabase through its REST API.

2. System architecture

The system has four main parts:

- Frontend PWA: static HTML, CSS, JavaScript modules, images, fonts, manifest, and service worker under public/.
- Supabase: recipe, ingredient, and relationship data; image storage; and the email Edge Function.
- Gmail: recipe email delivery through Gmail API OAuth, with authenticated SMTP as a fallback.
- Optional Cloudflare R2 recipe-share pages: public pages opened from generated QR codes.

The browser never receives Gmail credentials, the Supabase service-role key, or R2 private credentials. Sensitive values must remain in Netlify environment variables, Supabase secrets, or an administrator's local environment.

3. Repository layout

- public/index.html: application entry page.
- public/app.js: application state, navigation, language selection, idle handling, and recipe sharing.
- public/screens/: welcome, home, ingredients, results, recipe detail, and game screens.
- public/styles/: global, layout, component, PWA, and game styling.
- public/data/: Supabase REST access, local catalogue cache, configuration, email request, and QR utilities.
- public/i18n.js: native Slovenian and English interface translations.

- public/recipe-field-translations.js: locally stored English recipe descriptions, advice, methods, and numbered preparation steps.
- public/sw.js: offline shell and asset caching.
- public/pwa/: service-worker registration and kiosk interaction restrictions.
- public/assets/: images, icons, and fonts.
- supabase/functions/send-recipe-email/: server-side recipe email function.
- recipe-share/: standalone public recipe-share page.
- scripts/: environment, assets, images, icons, QR pages, and local-server tooling.
- netlify.toml: Netlify build, caching, and security-header configuration.

4. Local development

4.1 Requirements

- Node.js 20 or a current Node.js LTS release.
- npm.
- Access to the Supabase project.
- A modern Chromium-based browser for kiosk testing.

4.2 Install and start

```
npm install
npm run env:build
npm start
```

Open <http://localhost:4173>.

The frontend does not require a production bundling step. The build command generates the asset manifest and the public runtime environment file.

4.3 Public environment values

Create .env.local from .env.example. The browser needs only:

- SUPABASE_URL or VITE_PUBLIC_SUPABASE_URL.
- SUPABASE_ANON_KEY, containing the project's anon or publishable key.

Run `npm run env:build` after changing these values. This generates the gitignored public/env.js file.

Never place the following in public/env.js or any VITE_PUBLIC_* variable:

- Supabase service-role keys.
- Gmail passwords, OAuth client secrets, or refresh tokens.
- R2 secret access keys.

Every value shipped in the frontend can be read by a visitor.

5. Data model and content flow

The kiosk reads the Supabase tables recipes, ingredients, and recipe_ingredients. Public read access must be enabled only for the data required by the kiosk. Example policies are provided in supabase-kiosk-read-policies.sql.

On startup, the data store:

1. Loads the last valid catalogue from localStorage for a fast or offline start.

2. Fetches the current catalogue from Supabase.
3. Normalizes recipe fields, ingredients, servings, tags, and translated fields.
4. Refreshes the remote catalogue every five minutes by default.

The refresh interval can be changed with `ZDRAVO_REFRESH_INTERVAL_MS`.

Recipe and ingredient images normally come from public Supabase Storage buckets. Bundled images under `public/assets/images/` provide fallbacks.

6. Language system

Language switching is native, immediate, and does not reload the page. The selected locale is saved in `localStorage` under `zdravo.locale`.

The application does not load the Google Website Translator widget. No Google translation banner, badge, popup, or runtime script is shown to kiosk users.

English content comes from:

- Interface strings in `public/i18n.js`.
- Ingredient translations in `public/i18n.js`.
- Database fields such as `description_en`, `steps_en`, `dodatni_nasvet_en`, and `nacin_priprave_en`, when present.
- Local fallbacks in `public/recipe-field-translations.js`.

All 70 current recipes have local English preparation-step translations, covering 700 numbered steps.

Recipe names are intentionally never translated. They remain in their original Slovenian form in lists, detail headings, page titles, game references, and shared email content.

Language-specific images are selected by application state. English currently uses the English home hero, puzzle header, and detective header variants.

When adding a new recipe, prefer storing English fields in Supabase. If no English step list exists, add a `steps` array to the matching recipe entry in `public/recipe-field-translations.js`.

7. Recipe experience

The recipe detail screen displays:

- Original Slovenian recipe name.
- Localized description.
- Time, difficulty, servings, and suitability information.
- Localized ingredient names and units.
- Numbered, localized preparation steps.
- Localized additional advice with a pastel-green bulb icon.
- Email and QR sharing actions.

The servings controls recalculate displayed ingredient quantities. Recipe names remain unchanged when language or servings change.

The email success popup has a top-right close control and closes automatically after five seconds when left untouched.

8. Games

8.1 Farm to Plate puzzle

Players drag picture pieces from the side tray to their correct positions. Difficulty controls time, pre-placed pieces, distractors, points, and hint cost.

Touchscreen behavior:

- A vertical swipe scrolls the pieces tray with momentum.
- A sideways movement toward the board begins piece dragging.
- Overscroll is contained inside the tray.
- Hint and Restart stay in a fixed bottom action row.
- When solved, Complete replaces the Hint and Restart controls.
- The completed picture remains visible for ten seconds before the result screen.
- Complete can be pressed immediately to continue without waiting.

8.2 Market Detective

Players identify the correct ingredient or recipe answer. Correct and incorrect responses affect points according to difficulty. The correct-answer indicator uses a complete circular badge.

8.3 Replay behavior

Both games offer two replay paths:

- Continue Game preserves the accumulated score and starts another round.
- Start Again resets the score and starts a fresh game.

9. Sending recipes by email

Email is handled by the Supabase send-recipe-email Edge Function. The frontend sends structured recipe content to the function; it never sends Gmail credentials.

The function prefers Gmail API OAuth when all OAuth credentials are configured. If OAuth delivery fails and an app password is available, it falls back to authenticated Gmail SMTP on ports 465 and 587.

Configure Supabase secrets similar to:

```
supabase secrets set GMAIL_USER=...
supabase secrets set GMAIL_CLIENT_ID=...
supabase secrets set GMAIL_CLIENT_SECRET=...
supabase secrets set GMAIL_REFRESH_TOKEN=...
supabase secrets set GMAIL_APP_PASSWORD=...
supabase secrets set ZDRAVO_ALLOWED_ORIGINS=https://your-kiosk-domain
```

Then deploy:

```
supabase functions deploy send-recipe-email
```

The function creates multipart plain-text and HTML email content with standards-compliant MIME and CRLF line endings.

Gmail API delivery does not guarantee inbox placement. Spam classification also depends on sender reputation, message content, recipient behavior, and domain authentication. For a custom sending domain, configure SPF, DKIM, and DMARC. Do not repeatedly send identical test messages to many inactive recipients.

If OAuth reports `invalid_grant`, generate a new refresh token and update the Supabase secret. OAuth applications left in Google Cloud Testing mode commonly issue short-lived refresh tokens; publish the consent configuration for durable production use.

10. QR and public recipe sharing

The browser can generate a QR code for the current recipe. Optional public recipe pages are built from `recipe-share/` and can be synchronized to Cloudflare R2.

Useful commands:

```
npm run recipe-share:build
npm run recipe-qr:dry-run
npm run recipe-qr:sync
```

R2 access keys are private deployment credentials and must never be added to frontend files.

11. Offline operation and caching

The service worker precaches the application shell and progressively caches artwork. The last successful Supabase catalogue is also retained in `localStorage`.

This allows the kiosk to reopen and navigate during a temporary connection outage after it has been warmed once online.

The current service-worker cache version is `v20`. Increase `CACHE_VERSION` in `public/sw.js` whenever a cached shell file changes. Regenerate the asset list after adding or removing artwork:

```
npm run assets:manifest
```

After deployment, restart or refresh the installed kiosk once if it is still displaying an older cached interface.

12. Netlify deployment

The repository's `netlify.toml` defines:

- Build command: `npm run build`.
- Publish directory: `public`.
- Node.js version 20.
- Revalidation headers for `sw.js`, `env.js`, and `index.html`.
- Longer caching for fonts and artwork.
- Content Security Policy and related security headers.

Required Netlify environment values:

- `SUPABASE_URL`.
- `SUPABASE_ANON_KEY` using the anon or publishable key.

The current configuration contains `SECRETS_SCAN_ENABLED = "false"`. Disabling Netlify's scan does not make committed credentials safe. Verify that `.env.local`, generated `public/env.js`, service-role keys, Gmail credentials, and R2 secrets are excluded from version control. Rotate any secret that has been publicly exposed.

The site must be hosted at the domain root over HTTPS. Do not add a catch-all redirect that returns `index.html` for missing assets, because the service worker could cache HTML as an image or module.

13. Installing and operating the kiosk

1. Open the deployed HTTPS URL in Chrome or Edge on the kiosk computer.
2. Use the browser menu or address-bar install icon to install the PWA.
3. Launch the installed application.
4. Configure the operating system or browser kiosk mode for unattended operation.
5. Confirm touch dragging, tray scrolling, email delivery, SL/EN switching, sound, and offline startup.

The frontend blocks context menus, long-press menus, pinch zoom, and double-tap zoom. The first interaction can request fullscreen when the app is opened in a normal browser tab.

After two minutes without interaction, the app returns to the Welcome screen.

14. Common maintenance commands

<code>npm start</code>	Start the local static server on port 4173
<code>npm run build</code>	Build environment and asset manifest for deployment
<code>npm run env:build</code>	Generate public/env.js
<code>npm run assets:manifest</code>	Refresh the service-worker artwork manifest
<code>npm run icons:build</code>	Regenerate PWA icons
<code>npm run images:optimize</code>	Optimize new image assets
<code>npm run vendor:qrcode</code>	Rebuild the browser QR library
<code>npm run recipe-qr:dry-run</code>	Preview public recipe-page synchronization
<code>npm run recipe-qr:sync</code>	Publish public recipe pages
<code>npm run docs:pdf</code>	Regenerate this PDF manual

15. Troubleshooting

Application opens with no catalogue

- Confirm public/env.js exists in the deployed output.
- Confirm it contains an anon or publishable key, not a service-role key.
- Check Supabase public read policies and browser network errors.
- Confirm the configured Supabase project URL matches the key.

A deployed change is not visible

- Increase CACHE_VERSION in public/sw.js.
- Run `npm run assets:manifest`.
- Deploy again.
- Fully close and reopen the installed PWA, or clear the site's service-worker storage during testing.

English recipe details remain Slovenian

- Confirm the recipe ID has an entry in public/recipe-field-translations.js, or provide the corresponding *_en database field.
- Confirm steps_en is a valid array or JSON array.
- Keep the recipe title in name_sl; titles must not be translated.

Email returns HTTP 500

- Review Edge Function logs in Supabase.

- Confirm Gmail values are Supabase secrets, not only local Netlify values.
- Check OAuth refresh-token validity.
- Confirm the sender address matches the authenticated Gmail account.
- Confirm ZDRAVO_ALLOWED_ORIGINS includes the deployed kiosk origin.
- Redeploy the Edge Function after changing its source.

Puzzle controls or tray are inaccessible

- Confirm the kiosk has loaded service-worker cache v20 or later.
- Test at the actual kiosk resolution and browser scaling.
- Use a vertical swipe inside the pieces panel to scroll.
- Move sideways from a piece to initiate dragging.

16. Security checklist

- Keep .env.local out of Git.
- Never expose Supabase service-role credentials to the browser.
- Keep Gmail, OAuth, and R2 secrets in server-side secret stores.
- Use least-privilege Supabase Row Level Security policies.
- Restrict allowed email-function origins to production and approved test domains.
- Rotate exposed credentials immediately.
- Keep dependencies, browser, operating system, and Supabase CLI current.
- Review Content Security Policy when adding a new external service.
- Back up Supabase data and storage according to the project's recovery requirements.

17. Release checklist

Before each production release:

1. Review changes for accidentally committed secrets.
2. Run npm run build.
3. Run an ES-module bundle or syntax validation.
4. Verify SL and EN across every screen; recipe names must remain Slovenian.
5. Test both games with touch input.
6. Confirm Hint, Restart, and Complete remain visible at kiosk resolution.
7. Test puzzle-tray vertical scrolling and piece dragging.
8. Send a test recipe email and wait for the five-second success-popup close.
9. Test the QR page on a phone.
10. Test a warm offline restart.
11. Confirm the service-worker cache version was increased for shell changes.
12. Deploy and restart the installed kiosk once.

18. Support handover information

Maintain a private operational record containing the production URLs, Supabase project owner, Netlify site owner, Gmail account owner, OAuth project owner, R2 account owner, backup procedure, and credential-rotation schedule. Do not place actual passwords, keys, tokens, or recovery codes in this PDF.

Project name: Zdravo Jem Application type: Installable static PWA Primary target: 55-inch portrait touchscreen kiosk Frontend languages: Slovenian and English